

A project

CAMPUSCONNECT

A Unified Academic & Social Engagement Platform

CSE 400: Software Development Project

SUBMITTED BY

Name	ID	Intake
Tamjid Hossain Hridoy	20244103290	58th
Musfika Jannat	20244103293	58th
Farjana Haque	20244103308	58th
Md Asfaque Akhond	20244103288	58th
Nafisa Ulfat	20244103319	58th

SUPERVISED BY:

Md. Masudul Islam

Assistant Professor,
Department of Computer Science and Engineering

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
Bangladesh University of Business and Technology (BUBT)

**BANGLADESH UNIVERSITY OF BUSINESS AND TECHNOLOGY
(BUBT)**

Abstract

CampusConnect-Hub is a centralized web application designed to bridge the gap between academic management and campus social interaction within a single unified platform. Currently, university students rely on multiple disconnected tools — such as classroom management systems, social media applications, and messaging apps — to manage their academic and social lives. This fragmentation leads to information silos, missed opportunities, and a lack of verified, moderated communication.

This project proposes the development of CampusConnect-Hub using a Flask-based backend and PostgreSQL database, supported by a mobile-first frontend built with HTML5, Vanilla JavaScript, and Tailwind CSS. The system incorporates role-based access control (RBAC) for students and teachers, a centralized campus event feed, smart event discovery, an integrated academic classroom module, a real-time chat system, and a teacher-moderated content approval workflow.

The expected outcomes include improved student engagement, better campus communication, simplified academic access, and a more organized and verified campus environment. CampusConnect-Hub aims to reduce dependency on scattered external platforms and provide a clean, scalable, and secure solution tailored to the needs of university communities.

List of Figures

Sl. No.	Figure Name	Page
1	Use Case Diagram – CampusConnect	8
2	System Architecture Diagram	9
3	Context Level DFD (DFD-0)	10
4	Data Flow Diagram Level-1 (DFD-1)	11
5	Database Schema Diagram	12
6	System Flowchart	13

Table of Contents

Abstract.....i

List of Figures.....ii

Chapter 1 | Introduction.....1

1.1 Problem Specification/Statement.....1

1.2 Objectives.....1

1.3 Scope.....2

1.4 Organization of Project Report.....2

Chapter 2 | Background.....3-5

2.1 Existing System Analysis.....3

2.2 Supporting Literatures.....4

Chapter 3 | System Analysis & Design.....6-13

3.1 Technology & Tools.....6

3.2 Model & Diagram.....7

3.2.1 Development Model (Agile).....7

3.2.2 System Architecture.....8

3.2.3 Use Case Diagram.....9

3.2.4 Context Level Diagram.....10

3.2.5 Data Flow Diagram.....11

3.2.6 Database Schema.....12

3.2.7 Algorithms/Flowchart.....13

Chapter 4 | Implementation.....14-16

4.1 Interface/Front-End Design.....14

4.2 Back-End Development.....15

4.3 Modules/Features.....16

Chapter 5 | User Manual.....17-22

5.1 System Requirements.....17

5.2 User Interfaces.....18

Chapter 6 | Conclusion.....23-24

6.1 Conclusion.....23

6.2 Limitations.....23

6.3 Future Works.....24

References.....25

Appendix A.....26

Chapter 1

INTRODUCTION

1.1 Problem Specification/Statement

In modern universities, student activities are distributed across multiple disconnected platforms. Important academic notices are sent via email, event announcements are posted in Facebook groups, and academic materials are shared through separate classroom management systems. This fragmentation creates significant challenges: students miss critical events because they are not part of the right social group, club announcements are unverified and prone to misinformation, and there is no unified system that brings academic and social engagement under one roof.

Team 3 was motivated to design CampusConnect-Hub after observing how CSE students at BUBT routinely missed workshop registrations and club activities due to late or scattered announcements. The absence of a verified, role-based, and centralized system is the core problem this project addresses.

1.2 Objectives

The key objectives of CampusConnect-Hub are:

- To design and implement a unified web platform that integrates academic classroom management and campus social activity discovery.
- To develop a Role-Based Access Control (RBAC) system that separates student and teacher functionalities securely.
- To build a real-time integrated chat system for section groups, clubs, and event-based communication.
- To create a smart event discovery system that recommends events based on a student's department and interests.
- To implement a teacher-moderated content approval workflow that maintains content quality and authenticity.
- To deliver a mobile-first, responsive UI that provides a clean and efficient user experience.

1.3 Scope

CampusConnect-Hub is scoped for deployment within a university setting such as BUBT. Its scope includes:

- A centralized campus feed displaying events, club announcements, and university notices.

- An event management system allowing authorized users to create, manage, and register for events.
- An academic classroom module for teachers to upload materials and for students to join using unique codes.
- A built-in real-time chat system for section groups and club discussions.
- A basic AI assistant for answering academic queries and providing schedule information.
- The platform is limited to web-based access and is not natively deployed as a mobile app.

1.4 Organization of Project Report

This report is organized as follows:

- Chapter 2 – Background: Analyzes existing systems similar to CampusConnect and reviews supporting literature, tools, and methodologies.
- Chapter 3 – System Analysis & Design: Presents the development model, system architecture, use case diagram, context-level and data flow diagrams, database schema, and flowcharts.
- Chapter 4 – Implementation: Describes front-end interface design, back-end development, and the implemented modules.
- Chapter 5 – User Manual: Provides hardware/software requirements and user interface walkthroughs for different panels.
- Chapter 6 – Conclusion: Summarizes findings, outlines limitations, and proposes future work directions.

Chapter 2

BACKGROUND

2.1 Existing System Analysis

Several existing platforms partially address the needs of students in academic environments. A brief analysis of the most relevant ones follows:

Google Classroom

Google Classroom is a widely used learning management system that allows teachers to create classes, post assignments, and share materials. It excels in academic document sharing and assignment tracking. However, it lacks social engagement features such as event discovery, club management, or campus-wide announcements. It has no campus feed or notification system for non-academic activities.

Facebook Groups

Many universities rely on Facebook groups for club announcements and event sharing. While widely accessible, Facebook groups have significant drawbacks: posts are unmoderated and prone to spam and misinformation, non-members miss important announcements, academic and social content is mixed without any structure, and there is no role-based control for teachers or administrators.

Microsoft Teams

Microsoft Teams provides a robust communication and collaboration environment for academic institutions. It supports group chats, file sharing, and video conferencing. However, its complexity can be a barrier for students unfamiliar with enterprise tools, and it does not provide campus-specific features like event registration, smart discovery, or club management.

CampusConnect-Hub addresses the gaps in all three systems by combining academic management, social event discovery, moderated content, and integrated communication into a single platform designed specifically for university use.

2.2 Supporting Literatures

The development of CampusConnect-Hub draws on well-established software engineering knowledge, frameworks, and methodologies:

Flask (Python Web Framework)

Flask is a lightweight WSGI web application framework in Python, chosen for its simplicity, flexibility, and strong ecosystem. Flask-Login is used for session management and authentication. Its minimalism allows the team to build and scale features incrementally without framework overhead.

PostgreSQL

PostgreSQL is a robust, open-source relational database system known for strong ACID compliance, support for complex queries, and scalability. It is appropriate for handling the relational data structures in CampusConnect, including users, roles, events, classrooms, and messages.

Role-Based Access Control (RBAC)

RBAC is a well-studied security model in which access permissions are assigned to roles rather than individuals. This model ensures that students and teachers have clearly separated permissions, preventing unauthorized actions such as content publication without approval.

Tailwind CSS & Responsive Design

Tailwind CSS is a utility-first CSS framework that enables rapid UI development. Combined with a mobile-first design philosophy, it ensures the platform renders cleanly across all screen sizes, which is critical given that most students access university platforms via smartphones.

Agile Development Methodology

The project adopts an Agile approach, allowing iterative development with regular feedback loops. Features are developed in sprints, enabling the team to adapt to changing requirements and ensure that core modules are working before adding advanced features like the AI assistant.

Chapter 3

SYSTEM ANALYSIS & DESIGN

3.1 Technology & Tools

The following technologies and tools are used in the development of CampusConnect-Hub:

Category	Technology/Tool	Version/Notes
Frontend	HTML5, Vanilla JavaScript	Standard Web APIs
Styling	Tailwind CSS	v3.x, mobile-first
Backend	Flask (Python)	v2.x, WSGI
Database	PostgreSQL	v14+, Relational
Authentication	Flask-Login	Password hashing (bcrypt)
Version Control	Git / GitHub	Collaborative development
IDE / Editor	VS Code	Cross-platform
API Design	RESTful API	JSON responses

Hardware requirement: Minimum 4 GB RAM, modern multi-core processor, 10 GB storage.
Software requirement: Python 3.9+, Node.js (for Tailwind build), PostgreSQL 14+, a modern browser (Chrome, Firefox, Edge).

3.2 Model & Diagram

3.2.1 Development Model — Agile

The Agile software development model was selected for CampusConnect-Hub due to the iterative nature of the project. Development is divided into sprints, with each sprint targeting a specific set of features. Sprint 1 focuses on authentication and RBAC, Sprint 2 on the campus feed and event system, Sprint 3 on the classroom module and chat system, and Sprint 4 on the AI assistant and final integration.

Agile is beneficial here because requirements may evolve during development, and continuous testing ensures stable modules before new features are added. Daily stand-ups and sprint reviews within the 5-member team maintain coordination and accountability.

3.2.2 System Architecture

CampusConnect-Hub follows a three-tier architecture: Presentation Tier (HTML/CSS/JS frontend), Logic Tier (Flask backend with routing, authentication, and business logic), and Data Tier (PostgreSQL database). Client requests are handled by Flask routes, which interact with the database via an ORM layer (SQLAlchemy) and return JSON responses to the frontend.

[Figure 2: System Architecture Diagram — to be inserted here]

3.2.3 Use Case Diagram

The use case diagram captures the interactions between the two primary actors — Student and Teacher — and the system. Key student use cases include: Register/Login, View Campus Feed, Browse & Register for Events, Join Classroom, Participate in Chat, and View AI Assistant. Key teacher use cases include: Create Classroom, Upload Materials, Post Announcements, Approve Club Content, and Moderate Chat.

[Figure 1: Use Case Diagram — to be inserted here]

3.2.4 Context Level Diagram (DFD-0)

The context diagram shows CampusConnect-Hub as a single process interacting with four external entities: Student, Teacher, Club (authorized submitters), and Administrator. Students provide registration data and event registrations and receive feeds, event details, and classroom materials. Teachers submit academic content and receive approval workflows. Clubs submit event and post data pending teacher approval.

[Figure 3: Context Level Diagram — to be inserted here]

3.2.5 Data Flow Diagram (DFD-1)

The Level-1 DFD decomposes the system into its core processes: User Authentication, Event Management, Classroom Management, Chat System, and Content Moderation. Data stores include the User Database, Events Database, Classroom Database, and Messages Database. Data flows connect processes with their respective stores, showing how information is created, read,

and updated across the system.

[Figure 4: Data Flow Diagram Level-1 — to be inserted here]

3.2.6 Database Schema

The database consists of the following primary tables: Users (id, name, email, password_hash, role, department), Events (id, title, description, date_time, location, visibility, creator_id, approved), Classrooms (id, title, teacher_id, join_code), Enrollments (student_id, classroom_id), Messages (id, sender_id, room_id, content, timestamp), and EventRegistrations (student_id, event_id, registered_at). Foreign key relationships enforce referential integrity throughout.

[Figure 5: Database Schema Diagram — to be inserted here]

3.2.7 Algorithms/Flowchart

The primary flowchart describes the Event Discovery Algorithm: (1) Fetch logged-in user's department and interests. (2) Query Events table for upcoming approved events. (3) Apply a scoring function — events matching the user's department receive a higher relevance score. (4) Sort events by score descending. (5) Display top-N events as recommended in the feed. If no matches, fall back to displaying all upcoming events by date.

[Figure 6: Event Discovery Flowchart — to be inserted here]

Chapter 4

IMPLEMENTATION

4.1 Interface / Front-End Design

The frontend of CampusConnect-Hub is built using HTML5, Vanilla JavaScript, and Tailwind CSS. The design adheres to a mobile-first philosophy, ensuring that the interface is functional and visually consistent on both smartphone screens and desktop browsers.

The main interface elements include a bottom navigation bar on mobile with quick access to Feed, Events, Classrooms, and Chat sections. A card-based layout is used throughout the campus feed and event listings, making content easy to scan. Toast notifications provide real-time feedback to users after actions such as event registration or classroom join. The color palette and typography are kept minimal and clean to reduce cognitive load.

4.2 Back-End Development

The backend is developed using the Flask framework (Python). RESTful API endpoints handle all client-server communication. Flask-Login manages user sessions and authentication, while bcrypt is used for secure password hashing. SQLAlchemy serves as the ORM layer for database interactions with PostgreSQL.

Key backend modules include the Authentication Module (registration, login, logout, session handling), Event Module (CRUD for events, registration, approval workflow), Classroom Module (create/join classrooms, upload materials), Chat Module (real-time messaging using WebSockets or polling), and Recommendation Engine (department-based event scoring). All routes are protected by role decorators, ensuring that students cannot access teacher-only endpoints and vice versa.

4.3 Modules / Features

The following modules have been implemented in CampusConnect-Hub:

- Centralized Campus Feed — scrollable card-based feed showing events, notices, and club posts.
- Event Management System — create, view, register, and countdown for events with location and visibility settings.

- Smart Event Discovery — department-based event recommendation for personalized feeds.
- Academic Classroom Module — teacher-created classrooms with join codes, material uploads, and announcements.
- Role-Based Access Control (RBAC) — separate permissions and views for students and teachers.
- Teacher Approval System — club-submitted content held in a pending queue until teacher approval.
- Integrated Chat System — section group chats, club discussion groups, and event-based chat rooms.
- Notification & Feedback System — toast alerts, error messages, and action confirmations.
- Mobile-First Responsive Design — fully responsive layout with bottom nav for mobile users.
- AI Assistant (Demo) — basic assistant for answering academic queries and schedule lookups.

Chapter 5

USER MANUAL

5.1 System Requirements

5.1.1 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i3 or equivalent (1.6 GHz+)
RAM	4 GB (8 GB recommended)
Storage	10 GB free disk space
Network	Broadband Internet connection (for real-time features)
Display	1280 x 720 resolution or higher

5.1.2 Software Requirements

Software	Requirement
Operating System	Windows 10+, macOS 11+, or Ubuntu 20.04+
Python	Python 3.9 or higher
Database	PostgreSQL 14 or higher
Browser	Google Chrome 100+, Mozilla Firefox 100+, Microsoft Edge 100+
Node.js (optional)	v16+ for Tailwind CSS build step

5.2 User Interfaces

5.2.1 Panel A — Student Panel

Students access the platform via the Login page using their university email and password. Upon login, the Campus Feed is displayed as the home screen, showing recommended events and announcements sorted by relevance. Students can tap any event card to view full details and click Register. The bottom navigation bar provides quick access to the Classroom section (showing joined classes and materials), the Chat section (section group and club chats), and the Profile page.

[Screenshot: Student Feed Interface — to be inserted here]

[Screenshot: Event Registration Screen — to be inserted here]

5.2.2 Panel B — Teacher Panel

Teachers log in with role-specific credentials and are directed to a Teacher Dashboard. From the dashboard, teachers can create new classrooms (generating a unique join code), upload PDF or document materials to a classroom, post announcements, and access the Content Approval Queue where pending club submissions await review. Approved posts are immediately published to the campus feed; rejected submissions are removed with an optional feedback note.

[Screenshot: Teacher Dashboard — to be inserted here]

[Screenshot: Content Approval Queue — to be inserted here]

5.2.3 Login Credentials (Demo)

Role	Email	Password
Student	student@bubt.edu.bd	student123
Teacher	teacher@bubt.edu.bd	teacher123
Club Admin	club@bubt.edu.bd	club123

Chapter 6

CONCLUSION

6.1 Conclusion

CampusConnect-Hub is a practical, scalable, and student-centered solution that successfully combines the academic and social dimensions of university life into a single platform. Through this project, the team gained hands-on experience in full-stack web development, role-based system design, database modeling, and mobile-first UI/UX design.

The platform demonstrates that a Flask and PostgreSQL stack, when combined with a clean and responsive frontend, is capable of handling the complex needs of a university environment. The teacher moderation workflow and smart event discovery features, in particular, represent meaningful improvements over the current fragmented tools used by students at BUBT. The team believes CampusConnect-Hub can serve as a foundation for a fully deployed university platform.

6.2 Limitations

- The AI Assistant is in demonstration mode and relies on a simple rule-based system; it does not use a production-grade language model.
- Real-time chat is implemented using polling rather than a fully persistent WebSocket server, which may introduce latency under heavy traffic.
- The platform does not yet support native mobile app deployment (iOS/Android); it is web-only.
- File upload size is limited by server configuration; large academic documents (> 10 MB) may require additional backend optimization.
- The recommendation algorithm is basic; it does not incorporate machine learning or behavioral tracking.
- There is no email verification system for new user registrations in the current version.

6.3 Future Works

- Integrate a production-grade AI model (e.g., via Anthropic or OpenAI API) to power the academic assistant with contextual and course-specific answers.
- Develop native Android and iOS applications using React Native or Flutter for broader mobile reach.
- Implement WebSocket-based real-time chat to eliminate polling latency and support concurrent users.
- Add machine learning-based behavioral recommendation to the event discovery engine using collaborative filtering.

- Introduce email and SMS notification support for event reminders and classroom updates.
- Build an administrator panel for university IT staff to manage users, roles, and platform-wide settings.
- Extend the platform to support multiple universities with tenant-based separation.

References

1. "What Is a Data Flow Diagram." Lucidchart, www.lucidchart.com/pages/data-flow-diagram.
2. "Flask Web Development." Flask Documentation, flask.palletsprojects.com.
3. "PostgreSQL Documentation." The PostgreSQL Global Development Group, www.postgresql.org/docs.
4. "Use Case Diagram Tutorial." Creately, creately.com/guides/use-case-diagram-tutorial.
5. "System Architecture." InterviewBit, www.interviewbit.com/blog/system-architecture.
6. "Tailwind CSS Documentation." Tailwind Labs, tailwindcss.com/docs.
7. "What Is Agile?" Atlassian, www.atlassian.com/agile.
8. "Database Schema Design." Educative, www.educative.io/blog/what-are-database-schemas-examples.

Appendix A

This appendix contains supplementary materials for CampusConnect-Hub, including additional interface mockups, raw database schema SQL scripts, and API endpoint documentation that exceed the scope of the main report chapters.

A.1 Sample API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register a new user (student/teacher)
POST	/api/auth/login	Authenticate user and create session
GET	/api/events	Retrieve all approved upcoming events
POST	/api/events	Create a new event (authorized users only)
POST	/api/events/<id>/register	Register student for an event

Method	Endpoint	Description
GET	/api/classrooms	Get all classrooms for the current user
POST	/api/classrooms/join	Join a classroom using a unique code
GET	/api/chat/<room_id>/messages	Retrieve messages for a chat room